

* CrewAI -

◦ Crew - each department has agent → tools → tasks.

example: Trip planner agent

1. flight booking agent

2. hotel recommendation agent

3. sightseeing planner agent

4. food advisor agent

←
} just like crew members

◦ each CrewAI agent has:

◦ Name

◦ Role

◦ Goal

◦ Backstory

◦ Model

◦ Tools (APIs, db, etc)

example -



◦ new_scraper = Agent(
 name = "News Scraper",
 role = "Web Crawler",
 goal = "",
 backstory = "",
 llm = ChatOpenAI(model = "..."),
 tools = [news_scraper_tool]
)

◦ Process : workflow management system

↳ task :
assigned agent :
process type : Step 1 (Start)
↳ Sequential / Parallel

Practical

→ News Summarization

```
from crewai_tools import WebsiteSearchTool  
import os
```

```
web_search_tool = WebsiteSearchTool()
```

→ # LLM initiate

```
→ from crewai import Agent
```

```
researcher = Agent(role = 'Market Research Analyst',  
goal = ' ',
```

```
backstory = ' ',
```

```
tools = [web_search_tool],
```

```
) verbose = True
```

```
→ writer = Agent(role = 'Content Writer',
```

```
tools = [] # no need
```

```
)
```

agents, tools ✓

```
→ from crewai import Task, Crew
research = Task ( description = ' ',
                  expected_output = ' ',
                  agent = researcher
                )
```

```
→ write = Task ( "
                  agent = writer,
                  output_file = 'blog-posts/n_post.md' )
```

```
→ # Assemble the Crew
```

```
crew = Crew ( agents = [ researcher, writer ],
              tasks = [ research, write ],
              verbose = True,
              planning = True # enable AI planning
            )
```

```
crew.kickoff ()
```

Practical - 2

AI travel planner using CrewAI, langchain,
streamlit

```
→ from CrewAI import LLM
from langchain_ollama import OllamaLLM
llm = LLM ( model = 'ollama/llama3.2',
            base_url = "http://localhost:11434" )
```

```
→ # tool
```

```
from crewai.tools import tool
# Duck Duck Go Search Results
```

import json

@ tool

def search-web-tool (query : str):

search-tool = D (~ (num_results=10, verbose=True)

return search-tool.run(query)

Agent

→ from crewai import Agent

guide_expert = Agent (

role = " City local Guide Expert ",

goal = " _____ ",

backstory = " _____ ",

tools = [search-web-tool],

verbose = True

max_iter = 5,

llm = LLM (model=" ", base_url=" ")

allow_delegation = False,)

→ location_expert = Agent (~)

→ Planner^{expert}_Agent = Agent (~)

task

→ from datetime import datetime

« crewai import Task

location_task = Task (

description = f """
_____ """

expected-output = f """
_____ """

agent = location-expert,
output-file = 'city.md')

other tasks

→ guide-task = Task(_____)

→ planner-task = Task(_____)

→ # Now all the tasks should be written in
function as we ^{are} using Streamlit.

```
def location-task(agent, from-city, destination-city,  
                  date-from, date-to):
```

```
    return task(  
        description = f """  
        _____  
        _____ """
```

```
def guide-task(____):  
    return task(____)
```

```
def planner-task(____):  
    return task(____)
```

variables for calling function

→ location-task = location-task(location-expert,
 from-city,
 destination-city,

date - from,

date - to

)

guide_task = guide_task (~)

planner_task = planner_task (~)

→ from Crewai import Crew, Process

crew = (new (

agents = [d, g, p]

tasks = [~, ~, ~]

process = Process.sequential,

full_output = True,

share_crew = False,

) verbose = True

result = crew.kickoff()

Now in main.py x file name

→ from Travel Agents import guide_expert, d, g, p

u Travel Tasks u g-t, l-t, p-t

from crewai import Crew, Process

import streamlit as st

st.title (~)

st.markdown (~ ~ ~)

User Inputs

from_city = st.text_input("From City", "India")

des_city = " ("Destination City", "Rome")

date_from = st.date_input("Departure")

date_to = " ("Return")

interests = st.text_area(" ~ ")

Button to run CrewAI

if st.button(" ~ ") :

if not from_city or not des_city or not — :

st.error(" ~ ")

else :

st.write(" ~ ")

Initialize task

location_task = location_task(location_expert, —)

guide_task = guide_task(guide_expert, —)

planner_task = planner_task([location_task, guide_task], planner_expert, —)

Define Crew

crew = Crew (

agents = [location_expert, —, —],

tasks = [location_task, —, —],

process = Process.sequential,

```
full_output = True,  
verbose = True,
```

```
)
```

```
# Run Crew AI
```

```
result = crew.kickoff()
```

```
# Display
```

```
st.subheader(" — ")
```

```
st.markdown(result)
```

```
# result in string
```

```
travel_plan_text = str(result)
```

```
st.download_button(  
    label = " ",  
    data = travel_plan_text,  
    file_name = f" {dest-city}.txt",  
    mime = "text/plain"  
)
```

* Agno -

It is full-stack framework for building Multi-agent systems with memory, reasoning and knowledge.

Why?

- highly performant : very fast, very little memory.

- multi-modal : texts, image, audio, videos - input, o/p

↳ LangGraph - ^{by default} defaultly not a multi-modal.

but we can manually make it → issue

example : Audio transcription agent

1. use API for speech-text
2. create a node in langgraph for it
3. run the transcription and that text to other agent for summarization.

→ from agno. agent import Agent

Define Agno agent

```
agno_agent = Agent(  
    model = OpenAIChat(id="gpt-"),  
    description = "Agno Q/A agent",  
    markdown = False  
)
```

→ agno-agent . print_response ("What is India Capital",
stream = True)

* Create AI agents with tools and memory

→ from dotenv import load_dotenv
import os

load_dotenv(dotenv_path = " ")
api_key = os.getenv("OPENAI_API_KEY")
os.environ["O_"] = api_key

→ from agno.agent import Agent
from agno.models.openai import OpenAIChat
agent = Agent(model = OpenAIChat(id = " "),
description = " ",
markdown = True
)

→ agent.print_response("Summarize ", stream = True)

For tools, go to agno website

" tools option → check available
toolkits

example: Pre-built Tools

from agno.agent import Agent

from agno.tools.duckduckgo import DuckDuckGo

agent = Agent(tools = [], model = OpenAIChat(id = " "),
show_tool_calls = True,)

→ agent.print_response (—, stream = True)

example

→ from agno.agent import Agent

“ “ .models.openai import OpenAIChat

“ “ .tools.googlesearch import GoogleSearchTools

agent = Agent(model = OpenAIChat(id = “ “),

tools = [GoogleSearchTools(

stop_after_tool_call_tools = [“google-search”],
show_results_tools = [“google-search”],
show_tool_calls = True,

will show what the
result is

→ agent.print_response (—)

Custom Tools

→ from datetime import datetime

from agno.agent import Agent

def get_today_date() → str:

today = datetime.now().strftime(

“Today is %B %d, %Y”)

return today

get_today_date()

→ agent = Agent(tools = [get_today_date],

show_tool_calls = True,

markdown = True

```
agent.print_response("Today's date", stream=True)
```

example : Wikipedia API

```
→ import chitpa
```

```
from agno.agent import Agent
```

```
def s_wiki(topic: str = "Machine Learning") → str:
```

```
url = f"https://en.wikipedia.org/wiki/{topic.replace(' ', '%20')}"
```

```
response = chitpa.get(url, timeout=5.0)
```

```
data = response.json()
```

```
if "extract" in data:
```

```
title = data.get("title", topic)
```

```
extract = data["extract"]
```

```
return f"""**{title}**\n{extract}"""
```

```
else:
```

```
return "Sorry"
```

```
→ agent = Agent(tools=[s_wiki],
```

```
show_tool_calls=True,
```

```
markdown=True
```

```
)  
agent.print_response("", stream=True)
```

* Memory - Agno

3 types

1. Session Storage^a: stores the chat history of both user and agent.

2. User Memories (preferences): the agent stores insights that are learnt through the conversations with the user.
3. Session Summaries: chat history - too long, it stores the condensed version.

```
↳ from agno.agent import Agent
    " " " memory.v2.memory import Memory
    # extract imp info from user ↵
    " " " " " v2.db.sqlite import SQLiteMemoryDb
    # extracted memory will be stored in it ↵
from agno.storage.sqlite import SQLiteStorage
    # chat history ↵

from rich.pretty import pprint

db_file = "~.db"
user_id = "user-001"

memory = Memory( db = SQLiteMemoryDb
                  ( table_name = "user-memories",
                    db_file = db_file ),
                  )

agent = Agent( model = Qwen( ),
               description = "___",
               memory = memory,
               storage = SQLiteStorage( table, db_file ),
               enable_user_memories = True,
               add_history_to_messages = True,
```

```
num_history_recs=3, # last 3 msgs of user  
session_id="my-chat",  
markdown=True,
```

```
)  
→ agent.publish_response("", user_id=user_id)  
↓  
stream=True
```

To check user memories stored / not

```
→ print("In Current ** memories ** :")
```

```
user_memories = memory.get_user_memories(user_id=user_id)
```

```
pprint([{"memory": m.memory, "topics": m.topics}  
        for m in user_memories])
```

*Agentic RAG with Agno

⇒ What are the best laptops under 30,000?

1. Basic RAG app - searches a pre-stored VDB and writes an answer based on what it finds.

2. Agentic RAG APP -

- search Amazon, other websites, etc.
- filters by price
- compares specs (CPU, GPU, RAM)
- ranks 3 laptops.

Knowledge - domain-specific info that the Agent can search at runtime to make better decisions.

(dynamic few shot learning) and provide accurate responses (agentic RAG)

→ we have diff kind like PDF, text, JSON, CSV etc -
ArXiv, website.

* text knowledge base as sample 7.

→ from `agno.knowledge.text` import `TextKnowledgeBase`
" " `vectordb.pgvector` import `PgVector`

```
knowledge_base = TextKnowledgeBase(  
    path = " ",  
    vector_db = PgVector(  
        table_name = " ",  
        db_url = " ",  
    ),  
)
```

store in PostgreSQL

→ # use knowledge_base with agent
from `agno.agent` import `Agent`
from `knowledge_base` import `knowledge_base`
`agent = Agent(knowledge_base=knowledge_base,
 search_knowledge=True)`

→ # download and use PostgreSQL db

`psql --version`

install postgres

** remember pwd, port no.

→ # create DB

`psql -U postgres` → give password

set the path in environment variables.

* create ai database - in cmd prompt

CREATE USER ai with PASSWORD 'ai';

CREATE DATABASE ai; → grant permission to this user - database.

GRANT ALL PRIVILEGES ON DATABASE ai TO ai;

** install vector extension manually for PostgreSQL

* after data is fetched

→ agent.knowledge.load (recreate = False)

→ user_question = input("In ask question fromkbase")

→ agent.print_response (user_questions, user_id = "user-1", stream = True)